

Autonomy's Indexing file format (IDX)

Draft version 1.0

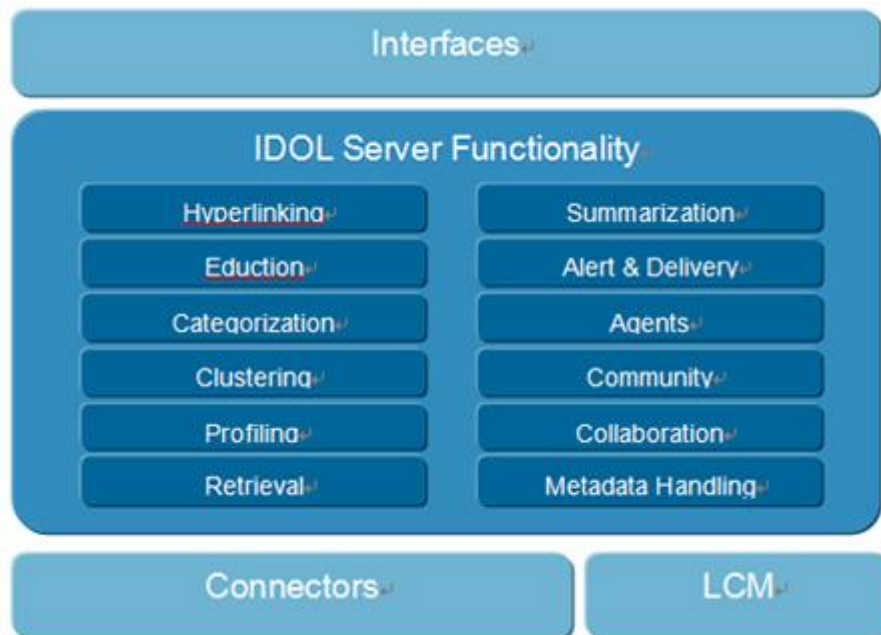
<i>Revision</i>	<i>Author</i>	<i>Description</i>
1 (4/9/2012)	Baron	Initial draft

Overview:

IDX 文件是 Autonomy 动态推理引擎 DRE (Dynamic Reasoning Engine) 使用的索引文件格式, 通过使用 IDX 索引文件, 在 Autonomy 的搜索技术框架的中间层-智能信息操作层 (IDOL), 整合了不同领域不同平台的信息到 Autonomy 搜索平台, 从而在 IDOL 的核心层次使用 DRE 实现了基于语义的搜索。关于 IDOL 以及 DRE 的简介如下:

IDOL (Intelligent Data Operating Layer), 即智能信息操作层, 是一种基础软件技术, 能够使企业自动处理各种讯息。IDOL 能够理解任何型式的讯息内容, 即理解后整合 (ITU), 实现了应用之间的相互通信, 同时无需手动创建复杂的连接界面或使用 MetaData。IDOL 不需要复杂的工作排程、繁杂的整合、商业逻辑或中间设备, 也不需要手工索引、连接或分类手续。其它应用程序可透过他们对非结构化讯息的共同理解迅速与 IDOL 整合, IDOL 则可以运用与人类相似的模式理解讯息 (它能够从人类可读的文档中读取内容, 直接关联概念, 而不倚赖由人工索引组成的规则)。

其基础框架如下图所示:



DRE (Dynamic Reasoning Engine), 即动态推理引擎, 它是智能数据操作层 (IDOL) 的核心, 是一种高度可扩展的多线程进程引擎, 可对目标内容进行分析并交付给用户。动态推理引擎基于先进的模式识别技术 (采用高性能机率模型技术)。

IDX 文件

一. IDX 文件的生成和使用方式

在 DRE 可以索引文件之前，数据需要作为 IDX 文件格式导入到 Autonomy 系统，按照 Autonomy 的文档的说明，获得 IDX 文件格式通常有三种方式：

1. 使用 connector。Autonomy 提供了许多 connector（如：AutoIndexer, HTTPFetch, Oracle Fetch, SharePoint Connector 等），这些 connector 可以从不同的数据源获得数据，并生成 IDX 格式数据。
2. 使用 DRE Administration Dialog。
3. 手动生成。手动创建一个 text 文件，这个文件以 DRE 特殊的字段组合而成。

当来自第三方平台的文档生成 IDX 文件格式以后，就可以使用 DRE 来索引了。我们可以通过以下的方式来使用 DRE：

1. 使用 Connector。一般 Autonomy 提供的 Connector 都有将 IDX 文件导入到 DRE 引擎的能力。
2. 使用 DRE Administration Dialog。
3. 使 HTTP 请求的方式直接导入。
4. 使用套接字连接，但是这个需要使用 POST 请求。

二. IDX 文件格式

IDX 文件从本质说来说是一个文本文件，这个文件包含可以导入到 DRE 引擎使用的 DRE 字段，DRE 字段规定了数据可以被 DRE 引擎索引的格式。

DRE 字段包含以下可能的值：

#DREREFERENCE

一个代表文档的独一无二的值，通常是文档名字，URL，或者文档的编码等。

#DRETITLE

文档的名字，可以有多行。

#DRECONTENT

文档的内容，可以有多行，此字段是可选的，如果没有指定此字段的值，需要执行多列#DREFIELD 的值，如果没有的话，则意味着文档没有内容

#DREFIELD Namen

预先指定的一些字段的名称，以名字="值"的形式给出。如果文档只包含一个字段，则可省略，即可使用#DRECONTENT代替。

#DRETYPE

文档的类型，只能是text类型。

#DREDATE

文件的创建日期，日期的格式是在DRE配置文件中指定的日期格式，默认是yyyy/mm/dd.

#DREDBNAME

存储索引数据的数据库名称

#DRESTORECONTENT

是否存储文件内容，y 代表存储，n 代表不存储。

#DRESECTION

段落标记，可以将一个大文档分为不同的部分。

#DREENDDOC

文档的结束标志。

注意：一个 IDX 文件必须以 **#DREREERENCE** 开头， 以 **#DREENDDOC** 结尾

一个 IDX 文本文件的内容示例如下：

```
#DREREERENCE 392348A0
#DREFIELD authornamel="Brown"
#DREFIELD authornamel="Edgar"
#DREFIELD title="Dr."
#DREDATE 1998/08/06
#DRETITLE
Jurassic Molecules
#DRECONTENT
Scientists announced last week the successful reproduction of a
possible precursor to all life on Earth. The molecules consist of a
part of DNA and the molecular "scissors" responsible for destroying
messenger RNA in humans.
Indexing documents into a DRE
Page 22
Using a technique called test tube evolution, scientists created a
nucleic acid enzyme, the first known enzyme that uses an amino acid
to start chemical activity. Scientists hope that the creation of this
molecule will lead to the elusive precursor. The precursor, by
definition, will have to contain both the genetic code for
replication and an enzyme to trigger self replication.
#DRETYPE text
#DREDBNAME Science
#DRESTORECONTENT y
#DREENDDOC
```

分节文档示例：

#DRREFERENCE 392348A0

#DREFIELD authorname1="Brown"

#DREFIELD authorname2="Edgar"

#DREFIELD title="Dr."

#DREDATE 1998/08/06

#DRETITLE

Jurassic Molecules

#DRESECTION 0

#DRECONTENT

Scientists announced last week the successful reproduction of a possible precursor to all life on Earth. The molecules consist of a part of DNA and the molecular "scissors" responsible for destroying messenger RNA in humans.

Using a technique called test tube evolution, scientists created a nucleic acid enzyme, the first known enzyme that uses an amino acid to start chemical activity. Scientists hope that the creation of this molecule will lead to the elusive precursor. The precursor, by definition, will have to contain both the genetic code for replication and an enzyme to trigger self replication.

At this point, no naturally occurring hybrid enzymes have been found. Scientists speculate that such enzymes may exist in nature and most certainly existed in Earth's early history.

#DRETYPE text

#DREDBNAME Science

#DRESTORECONTENT y

#DREENDDOC

#DRREFERENCE 392348A0

#DREFIELD authorname1="Brown"

#DREFIELD authorname2="Edgar"

#DREFIELD title="Dr."

#DREDATE 1998/08/06

#DRETITLE

Jurassic Molecules

#DRESECTION 1

#DRECONTENT

Scientists have known for some time that the key ingredients for life are DNA, RNA, and proteins. An interesting chicken-egg dilemma has developed: which came first, RNA, DNA, or proteins? Many believe that a replicating RNA molecule is the likely precursor to all life on Earth.

RNA serves as both a genetic molecule and an enzyme in the body, which scientists believe strongly suggests the likelihood of an RNA precursor to all life. They speculate that RNA was first, followed by

DNA, the much more stable of the two. It would serve as an efficient storehouse for the genetic code. Proteins, better catalysts than RNA, likely evolved later as well. At some point, the current three-based system developed from the initial one-based system of RNA.

Scientists hope that these scissors molecules may also have practical uses in medicine, since the molecules can efficiently shred specific DNA. Theoretically, it may be possible to tailor such a molecule to attack and shred harmful DNA from pathogenic organisms. These molecules could be made to be activated only in specific circumstances.

```
#DRETYPE text
#DREDBNAME Science
#DRESTORECONTENT y
#DREENDDOC
```

在每个 Section 开始时，以 0 排序开头，并且按照 DRE 的最佳实践，Section 部分的单词数最好不要超过 500，并且其后跟着#DRECONTENT 字段。

三. IDX 文件内容

结合项目中实际要生成的IDX文件，我们可以得到结论：最终的样式是由Autonomy的SharePoint Connector 2010生成的，其中其DRE配置文件定义了一些默认的字段名称和时间格式等，由于我们并没有获得DRE配置文件，因此根据我们获得IDX文件，我们大致总结其DRE配置文件中定义的**#DREFIELD Namen** 大致有以下一些值：

<i>AbsoluteApiUrl</i>	
<i>AUTN_COLLECTGROUP</i>	<i>Autonomy 定义的一些 DRE 字段，用于对 IDX 文件进行特殊的标识。</i>
<i>AUTN_GROUP</i>	
<i>AUTN_IDENTIFIER</i>	
<i>AUTONOMYMETADATA</i>	
<i>BaseSharePointType</i>	
<i>DisplayName</i>	
<i>DocTrackId</i>	
<i>FileTypeItemIdInList</i>	
<i>ListBaseTemplate</i>	
<i>ListBaseType</i>	

ListDescription	
ListGuid	
ListTitle	
ListUrl	
Name	
SiteDescription	
SiteGuid	
SiteGuidHierarchy	
SiteTitle	
SiteUrl	
Title	
Uniqueld	
UnixDateModified	
Url	
VersionId	
WebApplicationGuid	
WindowsDateModified	
X-DATASOURCE	Autonomy 定义的非标准字段
X-FILENAME	
X-ZANTAZ-PRIMARY-UID	

这些字段的大部分都是与 SharePoint 中的数据相关，其中值得注意的是以下几个值：

AUTN_COLLECTGROUP
 AUTN_GROUP
 AUTN_IDENTIFIER
 AUTONOMYMETADATA

这些是 Autonomy 自己定义的 DRE 字段数据，其中前两个与 Autonomy SharePoint Connector2010 配置相关，相对固定，根据已得到的 IDX 文件分析：

AUTN_COLLECTGROUP 一般为空，
 AUTN_GROUP 一般为 SP2010

AUTN_IDENTIFIER 字段标识的是一个文档的标识，是一个 Base64 编码的 xml 字符串，其中 xml 字符串的格式如下：

```
<id s="SHAREPOINT2010"
  r="http://win-2pv9b52ua22/Lists/VersionedList/DispForm.aspx?ID=1">
  <p n="ApiUrl" v="http://win-2pv9b52ua22/Lists/VersionedList/1_.000"/>
</id>
```

其中各个字符代表的意思可能如下：

Id 代表 Identifier 的简写形式，作为 xml 的节点名称。

- s 可能代表的是 Autonomy 的 connector 使用的一个配置文件的节点名字，一般情况下这个名字可能总是“SHAREPOINT2010”，但这个是 sharepoint connector 使用的名字，我们可能需要酌情修改。
- r 可能代表文档的引用值，可以是 URL 或者一个唯一标识。其中 url 一般来自浏览器中相类似的请求值等。
- n 可能代表 name 的意思，目前只能设置为“ApiUrl”
- v 可能代表 value 的意思，目前根据前面 name 中设置的 ApiUrl，其取值为 item 通过 API 取出的 URL

AUTONOMYMETADATA 字段根据目前的理解是关于权限相关的设置，是使用 Autonomy 软件中的 ACL（访问控制列表）使用的编码方式编码字符串，具体的加密方式还不是很清楚，AUTONOMYMETADATA 一个例子如下：

```
#DREFIELD AUTONOMYMETADATA="0:U:9urk60Hs7/Hg4v7+50P++ezh4ezp40Tjn/A:9v717P/o/eLk4/nx/vT++ejg8A:G:9onomJzom5nv6YCdmZSfgJmclZ+AlJyanICc6Z3p65+Vm+ic7Jvx7Pj54/7k+eju4uHh607550Lj70ng50P+8A:9onrmJ6Z65/u6ICe7JvpgJns7uyAlZjulYDsn5von0nrm53rm+zx7Pj54/3i/uT55Pvo+ujv7P394eTu7Pnk4uPw, 9onomJzom5nv6YCdmZSfgJmclZ+AlJyanICc6Z3p65+Vm+ic7Jvx++To+uj//vA:9onomJzom5nv6YCdmZSfgJmclZ+AlJyanICc6Z3p65+Vm+ic7Jvx+ej++Y3i+uPo//7w:9onomJzom5nv6YCdmZSfgJmclZ+AlJyanICc6Z3p65+Vm+ic7Jvx+ej++Y375P7k+eL//vA:9onomJzom5nv6YCdmZSfgJmclZ+AlJyanICc6Z3p65+Vm+ic7Jvx+ej++Y3g60Dv6P/+8A:NU::NG:9onrmJ6Z65/u6ICe7JvpgJns7uyAlZjulYDsn5von0nrm53rm+zx7Pj54+Po6uz55Pvo+ujv7P394eTu7Pnk4uPw"
```

注意示例中的红色背景的部分，有以下四种分隔符：

1. 0: U:
代表的是拥有权限的用户名称，中间可以使用逗号隔开，以区分更多的用户。
至于前面的 0，可能代表第一次出现的意思。
2. : G:
代表的是拥有权限的组的名称，中间可以使用逗号隔开不同的权限组。以区分更多的组。
3. : NU:
代表的是被明确排除权限的用户的名字，中间可以使用逗号隔开，以区分更多的用户。
4. : NG:
代表的是被明确排除权限的组的名字，中间可以使用逗号隔开，以区分更多的组。

在 IDX 文件中用于 AUTONOMYMETADATA 字段用户和组的信息使用 ACL（访问控制列表）使用的加密方式加解密，这种加密方式暂时还没有办法进行加解密。

还有几个值得关注的字段是以 X 开头的字段，根据我们对相关标准格式的研究，一般带有 X 开头的字段，都是属于没有成为广泛的标准，但通用程度较广的一类字段，如果 IDX 文件中的带有 X 的字段也是这种意图，那么我们可以有针对性的对其进行赋值。X 字段如

下：

X-DATASOURCE
X-FILENAME
X-ZANTAZ-PRIMARY-UID

结合相关的示例，X-DATASOURCE 可以认定为 IDX 文件的生成数据源，这个在 Autonomy SharePoint2010 Connector 导出的 IDX 文件中是 SP2010connector，我们也可以根据相应的规则进行赋值。X-FILENAME 可以认定 item 的 url 地址。X-ZANTAZ-PRIMARY-UID 可以认定为 ITEM 的创建者或者作者。虽然我们从示例 IDX 文件中认定了 X-ZANTAZ-PRIMARY-UID 的值，但是根据 ZANTAZ 这个单词的含义，我们猜测这个应该是一个公司的名字，因此从另外一个方面我们可以断定，这个应该是 DRE 配置文件中配置的针对这个公司的一个特殊的标识，由于没有最终确认，我们暂时在编码时做为预定义的字段处理。

除了以上这些字段之外，在第二节中定义的一些非 **namen** 型的字段在示例的 IDX 文件大部分都有涉及（如 DREDBNAME， DRETITLE 等等）。

除了以上字段之外，我们获得的 IDX 文件比较特殊的地方就是时间类型，前文已经指明，时间格式是在 DRE 配置文件中指定的格式，并且有默认的格式，根据我们目前掌握的格式，IDX 使用的时间格式应该是以下两种时间格式：

1. Unix 时间格式，Unix 时间格式是自公元 1970 年 1 月 1 日 0 时 0 分 0 秒以来以秒为单位的 64 位数值。
2. Windows file 时间格式，Windows file 时间格式是自公元 1601 年 1 月 1 日 0 时 0 分 0 秒以来以 100 毫微秒（或者说是 100 纳秒）为单位的 64 位数值。

除了以上信息，IDX 文件中使用的区别于常用文本格式的地方也是值得注意的。

IDX 文件没有文件头，这与一般我们常见的 Unicode 文本文件等有细微的区别，因此，我们不能使用常见的文本读写器来写文件，如 TextWriter 等一类的类，因为这些类一般都预置编码的。

还有一个不同是 IDX 文件每个 DRE 字段使用的换行符是 Unix 换行符，这与我们常用的 NewLine 换行符也是存在区别的。当然在每个 DRE 字段值的内部，其换行还是使用的回车加换行的形式。

四. IDX 生成的编码实现

为了实现针对上述认知的 IDX 文件的生成，并尽可能使用一些已有的代码结构进行整合，我们使用了定义在 GCommon 中的 MediaStorage 工程。

我们使用 AutonomyMetaData 类来定义 IDX 中存在的字段，如下定义：

```
public class AutonomyMetadata: Metadata

{

    [AutonomyMetadata("#DREREFERENCE", "")]
    public String DreReference { get; set; }
    [AutonomyMetadata("#DRETITLE", "")]
    public String DreTitle { get; set; }
    [AutonomyMetadata("#DREFIELD", "AbsoluteApiUrl")]
    public String AbsoluteApiUrl { get; set; }
    [AutonomyMetadata("#DREFIELD", "AUTN_COLLECTGROUP")]
    public String AutonomyCollectGroup { get; set; }
    [AutonomyMetadata("#DREFIELD", "AUTN_GROUP")]
    public String AutonomyGroup { get; set; }
    [AutonomyMetadata("#DREFIELD", "AUTN_IDENTIFIER")]
    public String AutonomyIdentifier { get; set; }
    [AutonomyMetadata("#DREFIELD", "AUTONOMYMETADATA")]
    public String AutonomyMetadataString { get; set; }
    [AutonomyMetadata("#DREFIELD", "BaseSharePointType")]
    public String BaseSharePointType { get; set; }
    [AutonomyMetadata("#DREFIELD", "ContentSize")]
    public String ContentSize { get; set; }
    [AutonomyMetadata("#DREFIELD", "DisplayName")]
    public String DisplayName { get; set; }
    [AutonomyMetadata("#DREFIELD", "DocTrackingId")]
    public String DocTrackId { get; set; }
    [AutonomyMetadata("#DREDATE", "")]
    public String DreDate { get; set; }
    [AutonomyMetadata("#DREDBNAME", "")]
    public String DreDatabaseName { get; set; }
    [AutonomyMetadata("#DREFIELD", "FileType")]
    public String FileType { get; set; }
    [AutonomyMetadata("#DREFIELD", "ItemIdInList")]
    public String ItemIdInList { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListBaseTemplate")]
    public String ListBaseTemplate { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListBaseType")]
    public String ListBaseType { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListDescription")]
    public String ListDescription { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListGuid")]
    public String ListGuid { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListTitle")]
    public String ListTitle { get; set; }
    [AutonomyMetadata("#DREFIELD", "ListUrl")]
}
```

```
public String ListUrl { get; set; }

[AutonomyMetaData("#DREFIELD", "Name")]
public String Name { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteCollectionGuid")]
public String SiteCollectionGuid { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteDescription")]
public String SiteDescription { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteGuid")]
public String SiteGuid { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteGuidHierarchy")]
public String SiteGuidHierarchy { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteTitle")]
public String SiteTitle { get; set; }
[AutonomyMetaData("#DREFIELD", "SiteUrl")]
public String SiteUrl { get; set; }
[AutonomyMetaData("#DREFIELD", "")]
public HashSet<MetaDataItemInfo> MetaDataItemInfoSet { get; set; }
[AutonomyMetaData("#DREFIELD", "Title")]
public String Title { get; set; }
[AutonomyMetaData("#DREFIELD", "UniqueId")]
public String UniqueId { get; set; }
[AutonomyMetaData("#DREFIELD", "UnixDateModified")]
public String UnixDateModified { get; set; }
[AutonomyMetaData("#DREFIELD", "Url")]
public String Url { get; set; }
[AutonomyMetaData("#DREFIELD", "VersionId")]
public String VersionId { get; set; }
[AutonomyMetaData("#DREFIELD", "WebApplicationGuid")]
public String WebApplicationGuid { get; set; }
[AutonomyMetaData("#DREFIELD", "WindowsDateModified")]
public String WindowsDateModified { get; set; }
[AutonomyMetaData("#DREFIELD", "X-DATASOURCE")]
public String XDataSource { get { return "DocAve6"; } }
[AutonomyMetaData("#DREFIELD", "X-FILENAME")]
public String XFileName { get; set; }
[AutonomyMetaData("#DREFIELD", "X-ZANTAZ-PRIMARY-UID")]
public String XZantazPrimaryUid { get; set; }
[AutonomyMetaData("#DRECONTENT", "")]
public String DreContent { get; set; }
}
```

每一个属性我们都使用一个 `AutonomyMetaDataAttribute` 来描述，以便使得其和 IDX 中的文件格式对应起来，这个类一旦定义，我们便把 IDX 整个文件的内容与格式的生成的逻辑

```
internal interface IAutonomyCompatibleForamFormatGenerator
{
    String Generate(MetaData metaData);
}
```

```

internal class AutonomyCompatibleFormatGenerator
    : IAutonomyCompatibleFormatGenerator
{
    public String SpaceChar { get { return " "; } }
    public String UnixNewLine { get { return "\n"; } }
    public String WindowsNewLine { get { return "\r\n"; } }

    public String Generate(MetaData metaData)
    {
        var stringBuilder = new StringBuilder();
        metaData.GetType().GetProperties().ForEach(property =>
        {
            var lineData = this.GetDataValue(property, metaData);
            stringBuilder.Append(lineData);
        });
        stringBuilder.AppendFormat("{0}{1}{1}", "#DREENDDOC", UnixNewLine);
        return stringBuilder.ToString();
    }

    String GetDataValue(PropertyInfo property, Object metaData)
    {
        var result = String.Empty;
        var attribute = property.GetAttribute<AutonomyMetaDataAttribute>();

        if (attribute != null)
        {
            var propertyValue = property.FastGetValue(metaData);
            if (propertyValue is IEnumerable<MetaDataItemInfo>)
            {
                var filedDataBuilder = new StringBuilder();
                var propertyRealValue = propertyValue as IEnumerable<Met
aDataItemInfo>;

                propertyRealValue.ForEach(item =>

```

```
{
    filedDataBuilder.AppendFormat(
        "{0}{1}{2}=\"{3}\"\"{4}\"",
        attribute.FieldName,
        SpaceChar,
        item.Name,
        item.Value,
        UnixNewLine);
    });
    result = filedDataBuilder.ToString();
}
else
{
    if (attribute.FieldName.EqualsIgnoreCase("#DREFIELD"))
        result = "{0}{1}{2}=\"{3}\"\"{4}\".FormatWith(
            attribute.FieldName,
            SpaceChar,
            property.Name,
            propertyValue.ToString(),
            UnixNewLine);
    else result = "{0}{1}{2}{3}".FormatWith(
        attribute.FieldName,
        SpaceChar,
        propertyValue.ToString(),
        UnixNewLine);
    }
}
return result;
}
}
```

代码相对来说比较简单,同时为了减轻我们在生成 IDX 文件中面临的特殊字段的问题,我们在 `AutonomyMetaData` 类中加入了生成 `Identifier` 的逻辑,如下面的方法所示:

```
String identifierTemplate = "<id s=\"{0}\" r=\"{1}\"><p n=\"{2}\" v=\"{3}\" /></id>";

public String GetAutonomyIdentifier(
    String referenceUrl,
    String apiUrl,
    String sectionName = "SHAREPOINT2010",
    String navigate = "ApiUrl")
{
    }
```

```
var identifier = identifierTemplate.FormatWith(sectionName, referenceUrl, navigate, apiUrl);  
var identifierArray = Encoding.UTF8.GetBytes(identifier);  
return Convert.ToBase64String(identifierArray);  
}
```

在 GCommonExtension 工程中，我们拓展了 DateTime 类型，添加了直接转换为 Unix 时间和 WindowsFile 时间的方法，如下所示：

```
public static Int64 ToUnixTime(this DateTime dateTime)  
public static Int64 ToWindowsFileTime(this DateTime dateTime)
```

这样，通过以上工作，我们基本上完成了 IDX 文件的初步生成工作。